



université PARIS-SACLAY

UNIVERSITÉ VERSAILLES SAINT-QUENTIN

Projet en langage C

Implémentation de schémas à seuil basés sur la signature RSA et le chiffrement El Gamal

Matteo Soreda & Quentin Mareau

Année 2025-2026

Table des matières

I	Introduction	1
II	Protocole de signature à seuil RSA	3
II.1	Préliminaires mathématiques	3
II.2	Présentation du protocole	5
II.3	Étude de la sécurité	7
III	Protocole de chiffrement à seuil El Gamal	10
III.1	Présentation du protocole	10
III.2	Etude de la sécurité	11
IV	Implémentation	12
V	Performances de l'implémentation	14

I Introduction

L'objectif de ce projet est de présenter deux protocoles cryptographiques à seuil. L'un basé sur la primitive RSA pour de la signature à seuil et l'autre sur la primitive du logarithme discret pour un chiffrement à seuil. Bien qu'ayant presque 50 ans, ces deux primitives asymétriques sont aujourd'hui toujours considérées comme sûres et efficaces, du moins jusqu'à l'éventuelle arrivée d'ordinateurs quantiques ou de nouveaux algorithmes.

Le plan de ce rapport est le suivant : Après une courte introduction rappelant le principe de la signature basée sur la primitive RSA, du chiffrement El Gamal, le principe d'un schéma de partage de secret ainsi que celui d'une signature à seuil, nous nous intéresserons à l'étude des deux protocoles annoncés. On présentera alors quelques théorèmes clés sur lesquelles reposent la théorie de ces deux protocoles, suivis de ces dits protocoles et de preuves de sécurité. Ce projet se voulant également pratique, nous proposerons ensuite une implémentation en C qui sera détaillée en partie IV.

Notions de bases : Une signature (numérique) est un procédé cryptographique asymétrique ayant principalement deux objectifs celui d'assurer l'authenticité et l'intégrité d'un document. Cela signifie que le signataire, de part l'utilisation de sa clé privé dans le mécanisme de signature, prouve à la fois que le texte signé est bien celui qu'il a envoyé (pas de modification par une tierce personne possible), mais cela empêche aussi le signataire de nier dans le futur d'avoir envoyé ce texte (on parle de non-répudiation). Bien qu'il existe des signatures post-quantiques de plus en plus utilisés (DILITHIUM, FALCON, etc), aujourd'hui encore celles qui prédominent sont celles qui ont résistés à des dizaines d'années de cryptanalyses notamment les primitives RSA et EL GAMAL.

La signature RSA repose sur le paradigme « hash and sign ». Un message M est donc d'abord haché pour obtenir un texte $x = H(M)$. Ce dernier est ensuite bourré de 0 et de 1, d'une façon fixé à l'avance selon le protocole choisit, de sorte à obtenir un texte ayant à peu près le même nombre de bits que le module RSA c'est-à-dire 1024 ou 3000 bits selon la sécurité souhaitée. On note β la fonction de bourrage et $\mu = \beta \circ H$ de sorte à ce que $\mu(M) = \beta(x)$ soit le texte inintelligible de 1024 ou 3000 bits. Il ne reste plus qu'à lui appliquer la primitive RSA pour obtenir la signature du message M , c'est-à-dire que le signé de M est $\mu(M)^d \bmod n$ où n est le module RSA et d l'inverse modulo $\lambda(n)$ de l'exposant public du signataire, avec λ la fonction indicatrice de Carmichael.

Le chiffrement El-Gamal consiste à exploiter la difficulté à calculer un logarithme discret dans certains groupes cycliques. Par exemple Bob se place dans un corps $\mathbb{F}_q$ où q est une grande puissance d'un nombre premier. Il choisit $g \in \mathbb{F}_q^\times$ générateur (c'est-à-dire un élément d'ordre $q-1$) et b un entier tiré aléatoirement entre 0 et $q-2$. Sa clé publique est alors la famille $(\mathbb{F}_q, g, g^b)$ et sa clé privé est b . Si Alice veut lui envoyer un message $M \in \mathbb{F}_q^\times$ elle va tirer aléatoirement un entier a entre 0 et $q-2$ et calculer à partir de g l'élément g^a et à partir de g^b l'élément g^{ab} . Elle envoie alors à Bob le couple $(g^a, g^{ab}M)$. Pour déchiffrer Bob n'a qu'à utiliser g^a et b pour

calculer g^{ab} puis son inverse et le multiplier par $g^{ab}M$ pour récupérer M .

L'objectif d'un schéma de partage de secret est de répartir une donnée secrète (par exemple un mot de passe ou bien la clé secrète d'un schéma asymétrique) entre plusieurs personnes (on dit qu'ils possèdent des clés secrètes de partage) de sorte à ce qu'il faille réunir un certain nombre de ces personnes pour retrouver ou simuler cette donnée, et que si ce nombre n'est pas atteint, aucune information sur les clés secrètes de partage n'est obtenables, même pour les personnes du groupe (on parle de non-forgeabilité du schéma). Précisément, si on se donne un groupe de l personnes et k un entier non nul plus petit que l , un partage de secret à seuil $(k; l)$ nécessite l'utilisation d'au moins k clés secrètes de partage pour retrouver la donnée secrète ou pour chiffrer/signer un message selon l'objectif. Mais un ensemble de $t < k$ personnes de ce groupe ne pourrait ni chiffrer, ni signer, ni obtenir d'information sur les clés secrètes de partage des $l - t$ autres personnes du groupe ou sur la donnée secrète. Un exemple classique pour $k = l$ où l'on cherche à retrouver la donnée secrète est, à l'instar d'un One Time Pad, de se reposer sur le XOR. Toutes sauf une clés secrètes de partage sont alors des nombres aléatoires de n bits, tandis que la dernière personne possède comme clé secrète partagée la somme XOR de la donnée secrète et des autres clés secrètes de partage. Dans un cadre plus général, on peut compter sur le schéma de partage de secret de Shamir à seuil $(k; l)$ qui est flexible et efficace. Il repose sur le principe d'interpolation. On se donne un corps finis K et un secret $a_0 \in K$, qui représente par exemple une clé de 256 bits qui a été utilisée pour protéger via un AES une donnée confidentielle comme les résultats d'un vote, et que l'on voudrait répartir entre 3 représentants de sorte à ce que si l'un d'entre eux perd sa part ou n'est pas disponible au moment du dépouillement, on puisse quand même déchiffrer le résultat du vote (c'est-à-dire on veut un schéma de partage à seuil $(k; l) = (2; 3)$). Le schéma de Shamir consiste alors à tirer dans K au hasard des valeurs $a_1, \dots, a_{k-1}$, à définir le polynôme $f = \sum_{i=0}^{k-1} a_i X^i \in K[X]$ et à donner à chacun des l représentants un couple $(i, f(i))$ pour $1 \leq i \leq l$. Ainsi, connaissant k tels couples on peut retrouver f comme l'unique polynôme interpolateur de degré inférieur à $k - 1$ et on obtient finalement la clé secrète $a_0 = f(0)$ comme étant son terme constant. Le fait que les a_i soient tirés au hasard et que le polynôme interpolateur existe et soit unique permet de garantir la non-forgeabilité du schéma.

II Protocole de signature à seuil RSA

II.1 Préliminaires mathématiques

On considère dans ce rapport $n = pq$ avec p, q deux nombres premiers. Ainsi, puisque n n'est pas premier, $\mathbb{Z}_n^\times$ n'est pas cyclique. Cependant, il est possible d'exhiber un sous-groupe cyclique de $\mathbb{Z}_n^\times$ facile à manipuler et qui a la bonne propriété d'être d'ordre très grand.

Lemme II.1. Soient p, q deux nombres premiers distincts tels que $p = 2p' + 1$ et $q = 2q' + 1$ avec p', q' deux nombres premiers impairs. On pose $n = pq$ et $m = p'q'$.

Alors $\mathbb{Z}_n^\times \simeq \mathbb{Z}_m \times \mathbb{Z}_2 \times \mathbb{Z}_2$ où le groupe de droite est muni de la loi produit induite par les lois additives usuelles.

Preuve. Par théorème des restes chinois,

$$\mathbb{Z}_n^\times \simeq \mathbb{Z}_p^\times \times \mathbb{Z}_q^\times \simeq \mathbb{Z}_{p-1} \times \mathbb{Z}_{q-1}$$

Puis comme $p - 1 = 2p'$ et $q - 1 = 2q'$ avec p', q' impairs (et distincts) on obtient, à nouveau par restes chinois :

$$\mathbb{Z}_n^\times \simeq \mathbb{Z}_{p'} \times \mathbb{Z}_{q'} \times \mathbb{Z}_2 \times \mathbb{Z}_2 \simeq \mathbb{Z}_m \times \mathbb{Z}_2 \times \mathbb{Z}_2$$

□

Maintenant que l'on connaît la structure de $\mathbb{Z}_n^\times$, il est facile d'en exhiber un sous-groupe cyclique :

Proposition II.2. Avec les mêmes notations, le sous-groupe Q_n des carrés de $\mathbb{Z}_n^\times$ est un groupe cyclique d'ordre m .

Preuve. On note $\phi : \mathbb{Z}_n^\times \longrightarrow \mathbb{Z}_m \times \mathbb{Z}_2 \times \mathbb{Z}_2$ l'isomorphisme du lemme précédent, explicitement donné par le théorème des restes chinois. On remarque que $\phi(Q_n) = \mathbb{Z}_m \times \{0\} \times \{0\}$. Il est clair que $\phi(Q_n) \subset \mathbb{Z}_m \times \{0\} \times \{0\}$. Pour le sens réciproque il suffit de remarquer que 2 est inversible modulo m . Ainsi, tout élément z à droite s'écrit $z = (2y, 0, 0)$. En notant x l'unique antécédent de $(y, 0, 0)$ par ϕ , on a $\phi(x^2) = z$. En particulier, Q_n est isomorphe à $\mathbb{Z}_m$ et est donc cyclique d'ordre m . □

Pour mettre en place les protocoles cryptographiques nous allons avoir besoin de générateurs du groupe cyclique Q_n . Il paraît alors important de pouvoir estimer leurs nombres :

Corollaire II.3. Avec les mêmes notations, Q_n est engendré par $p'q' - (p' + q' - 1)$ éléments.

Preuve. Quitte à considérer un isomorphisme il suffit de prouver que $p'q' - (p' + q' - 1)$ éléments engendrent Z_m . Or en notant g un générateur de ce groupe cyclique on remarque facilement que les générateurs de Z_m sont exactement les $g \times u$ avec u inversible. Il y a donc autant de générateurs que d'éléments inversibles. Or les éléments non inversibles sont : $0, p', 2p', \dots, (q' - 1)p', q', 2q', \dots, (p' - 1)q'$ au nombre de $p' + q' - 1$. □

La proportion d'éléments de Q_n non générateurs est alors $\frac{p' + q' - 1}{p'q'}$. Ainsi, si $p' < q'$, on a :

$$\frac{p' + q' - 1}{p'q'} \leq \frac{2q'}{p'q'} \leq \frac{2}{p'}$$

On pourra donc majorer cette proportion par $\frac{2}{p'}$ dans la suite. Ainsi, si p' est de l'ordre de 512 bits, on considère cette proportion comme négligeable.

Dans les protocoles suivants, la mise en commun des k clés secrètes de partage pour retrouver une $(k + 1)$ -ème valeur repose sur de l'interpolation polynomiale. Le point clé est que pour des nombres premiers p', q' de taille cryptographique, tout se passe comme lors d'une interpolation de Lagrange sur un corps, puisque $\mathbb{Z}_m$ se comporte comme tel, le nombre d'éléments non-inversible étant négligeable. Précisément :

Proposition II.4. Soit k un entier non nul, p', q' deux nombres premiers distincts et $m = p'q'$. Soient $x_0, x_1, \dots, x_{k-1}, y_0, \dots, y_{k-1} \in \mathbb{Z}_m$ tels que pour tout i, j dans $\{0, \dots, k-1\}$ distincts, $x_i - x_j$ est inversible dans $\mathbb{Z}_m$. Alors il existe un unique polynôme P sur $\mathbb{Z}_m$ de degré au plus $k-1$ tel que pour tout i dans $\{0, \dots, k-1\}$ on a $P(x_i) = y_i$.

Preuve. Soit $P = \sum_{i=0}^{k-1} a_i X^i$. On note A (resp. Y) le vecteur colonne de taille k de coordonnées les a_i (resp. les y_i). On a alors :

$$(\forall j \in \{0, \dots, k-1\}, P(x_j) = y_j) \iff \text{Vand}(x_0, \dots, x_{k-1})A = Y$$

où $\text{Vand}(x_0, \dots, x_{k-1})$ est la matrice de Vandermonde associée aux $x_0, \dots, x_{k-1}$. Ainsi l'existence et l'unicité d'un tel polynôme P sont équivalentes à l'inversabilité de cette matrice. Or, on rappelle que $\det(\text{Vand}(x_0, \dots, x_{k-1})) = \prod_{0 \leq i < j \leq k-1} x_j - x_i$ ce qui est bien inversible dans $\mathbb{Z}_m$ par hypothèse, comme produit d'éléments inversibles. $\square$

En pratique on connaît le polynôme interpolateur. En gardant les notations précédentes il s'agit de $P = \sum_{i=0}^{k-1} L_i y_i \in \mathbb{Z}_m[X]$ où $L_i = \frac{\prod_{j \neq i} X - x_j}{\prod_{j \neq i} x_i - x_j} \in \mathbb{Z}_m[X]$. Cependant en pratique on est confronté à deux problèmes : On est pas sûr que le dénominateur soit inversible et seul le dealer connaît m , donc seul lui peut faire les inversions modulo m . Pour résoudre ces problèmes on multiplie P par un facteur constant. Précisément on a le lemme suivant :

Lemme II.5. Soit $l, k \in \mathbb{N}$, $k \leq l$, $x_0, \dots, x_{k-1}, y_0, \dots, y_{k-1} \in \{0, \dots, l\}$ où les x_i sont 2 à 2 distincts et $P = \sum_{i=0}^{k-1} L_i^S y_i \in \mathbb{Q}[X]$ où $S = \{x_0, \dots, x_{k-1}\}$ et $L_i^S = \frac{\prod_{u \in S \setminus x_i} X - u}{\prod_{u \in S \setminus x_i} x_i - u} \in \mathbb{Q}[X]$. On se donne $x \in \{0, \dots, l\} \setminus S$. Alors

$$l!P(x) = \sum_{i=0}^{k-1} \lambda_i^S(x) y_i \in \mathbb{Z}$$

où $\lambda_i^S(x) := l!L_i^S(x) \in \mathbb{Z}$.

Preuve. La seule chose à démontrer ici est que $\lambda_i^S(x) = \frac{l!}{\prod_{j \neq i} x_i - x_j} \prod_{j \neq i} x - x_j \in \mathbb{Z}$. On a

$$\prod_{j \neq i} x_i - x_j = \prod_{\substack{j \neq i \\ x_j \leq x_i}} x_i - x_j \prod_{\substack{j \neq i \\ x_j > x_i}} x_i - x_j$$

Dans le premier produit les termes sont tous différents (car les x_j sont 2 à 2 distincts), positifs et inférieurs à x_i . Ce produit divise donc $x_i!$. Le deuxième produit vaut au signe près

$$\prod_{\substack{j \neq i \\ x_j > x_i}} x_j - x_i$$

dont les termes sont tous différents, positifs et inférieurs à $l - x_i$. Ce produit divise donc $(l - x_i)!$. Ainsi $\prod_{j \neq i} x_i - x_j$ divise $x_i!(l - x_i)!$ qui divise $l!$. Finalement $\lambda_i^S(x)$ est un entier comme produit de deux entiers. □

II.2 Présentation du protocole

On suppose pour ce protocole que $k = t + 1$, où l'on rappelle que t est le nombre de joueur corrompu. Autrement dit, connaître tous les secrets de partage sauf un, ainsi que toutes les données échangées lors du protocole, ne doit donner aucune information sur le secret de partage du dernier joueur ni permettre de signer sans lui. Nous allons maintenant présenter le rôle du dealer et des joueurs dans la mise en place du protocole.

Le dealer : il choisit deux nombres premiers p et q qui resteront secrets. Pour assurer la sécurité du protocole, ceux-ci doivent posséder les mêmes propriétés que pour n'importe quel protocole se basant sur RSA. En particulier, on impose qu'ils soient de taille 512 bits, ce qui assure une sécurité en 2^{80} d'après les meilleurs algorithmes de factorisation connus, qui sont sous-exponentiels.

De plus, on impose que $p = 2p' + 1$ et $q = 2q' + 1$ avec p' et q' deux nombres premiers. On pose alors $n = pq$ (qui sera notre module RSA) et $m = p'q'$. Enfin, il choisit un nombre premier $e > l$ différents de p' et q' , et calcule $d \in \mathbb{Z}$ tel que $ed \equiv 1 \pmod{m}$. À noter qu'un tel d existe car e est premier avec m .

Proposition II.6. Pour tout $x \in Q_n$, le sous-groupe des carrés de $\mathbb{Z}_n^\times$, on a $(x^e)^d = x \pmod{n}$.

Preuve. Soit $x = y^2 \in Q_n$ et k l'entier tel que $ed = 1 + km$. En notant λ la fonction indicatrice de Carmichael on a :

$$\lambda(n) = \text{ppcm}(p-1, q-1) = 2p'q' = 2m$$

Ainsi,

$$(x^e)^d = y^{2ed} = y^{2+2km} = x \times (y^k)^{2m} = x$$

□

Comme pour un RSA classique, la clé publique du dealer est alors constituée du couple (n, e) . Ensuite, le dealer fixe $a_0 = d$ et choisit aléatoirement $a_1, \dots, a_{k-1}$ dans $\mathbb{Z}_m$. On pose alors :

$$f = \sum_{i=0}^{k-1} a_i X^i \in \mathbb{Z}[X]$$

Il poursuit en calculant pour tout $i \in \{1, \dots, l\}$:

$$s_i = f(i) \mod m$$

Enfin, le dealer choisit aléatoirement $v \in Q_n$, le sous-groupe des carrés de $\mathbb{Z}_n^\times$. On rappelle qu'à la suite du corollaire II.3, nous avons conclu que la probabilité d'obtenir un élément de Q_n non générateur était de l'ordre de $\frac{1}{2^{512}}$. Il est alors raisonnable d'estimer qu'un élément $v \in Q_n$ choisi aléatoirement est un générateur. Ensuite, pour tout $i \in \{1, \dots, l\}$, il calcule :

$$v_i = v^{s_i} \in Q_n$$

Pour chaque joueur i , le nombre s_i est sa **clé secrète de partage** et v_i sa **clé de vérification**. Le dealer transmet alors les clés de partage (la clé privé du joueur) et de vérification (la clé publique du joueur) à chaque joueur de façon supposée sécurisée. De plus, on dira que v est la **clé de vérification du dealer**, également publique.

Soit M le message que l'on veut signer. On pose $x = H(M)$, avec $H : \{0, 1\}^* \longrightarrow \mathbb{Z}_n^\times$ une fonction de hachage. On cherche alors à combiner les informations de k joueurs intègres afin de construire $y \in \mathbb{Z}_n^\times$ tel que $y^e = x$, comme lors d'une signature RSA classique.

Génération de signatures partielles : chaque joueur i va maintenant générer une signature partielle $x_i = x^{2^{l!}s_i} \in Q_n$ où x est toujours le haché du message. Il voudra ensuite communiquer cette signature partielle à un coordinateur qui signera ultimement le message. On va alors accompagner chacune des signatures partielles d'une preuve d'exactitude zero-knowledge.

Pour se faire, notons $x' = x^{4^{l!}} \in Q_n$ de sorte que $x_i^2 = (x')^{s_i}$. Le joueur i tire alors aléatoirement un entier r de $L(n) + 2L_1$ bits où $L(n)$ est le nombre de bits de n et L_1 est un paramètre de sécurité (par exemple 128) sur lequel on reviendra dans la partie étude de sécurité. Il s'engage ensuite sur la valeur de r en calculant $c = H'(v, x', v_i, x_i^2, v^r, x'^r)$ où $H' : \{0, 1\}^* \longrightarrow \{0, 1\}^{L_1}$ est une fonction de hachage. À noter que dans cette fonction de hachage, les 3 premiers arguments sont publics, le 4ème est envoyé (et donc récupérable sur un canal non sécurisé) lors de la phase de mise en commun, tandis que les deux derniers ne sont calculables que par le joueur i . On remarque également que cette procédure permet au joueur i d'à la fois signer x' (et donc x car le nombre de racine $4^{l!}$ -ième d'un élément aléatoire de Q_n est négligeable devant $m = \text{Card}(Q_n)$), de prouver qu'il connaît s_i et que $x_i^2 = (x')^{s_i}$ (grâce à la présence des clés de vérification en argument de la fonction de hachage). De là, le joueur i envoie donc avec le triplet (x_i, z, c) où $z = s_i c + r$.

Vérification de l'authenticité de chaque joueur : Le coordinateur désigné veut vérifier que le triplet issu de chaque joueur est authentique. On note $S = \{i_1, \dots, i_k\} \subset \{1, \dots, l\}$ l'ensemble des joueurs associés à ces signatures partielles. Soit $i \in S$ et (u_i, z, c) le triplet reçu. La personne vérifie que $H(v, x', v_i, u_i^2, v^z v_i^{-c}, x'^z u_i^{-2c}) = c$. Si c'est le cas, cela signifie que $u_i^2 = (x')^{s_i} (= x_i^2)$ et on dit que la signature du joueur i est valide.

Mise en commun : On suppose que le coordinateur désigné dispose de k signatures valides. Pour $i \in S$, il dispose donc de $x_i \in \mathcal{Q}_n$ tel que $x_i^2 = x^{4l!s_i}$. Il calcule alors $w = \prod_{i \in S} x_i^{2\lambda_i^S(0)}$ où les λ_i^S sont ceux du lemme II.5. Il calcule ensuite w^e qui vaut :

Lemme II.7. Avec les notations précédentes, $w^e = x^{4(l!)^2} \mod n$

Preuve. On a $w = \prod_{i \in S} x_i^{2\lambda_i^S(0)} = \prod_{i \in S} (x^{4l!s_i})^{\lambda_i^S(0)} = x^{4l!\sum_{i \in S} s_i \lambda_i^S(0)}$. Or $s_i = f(i) \mod m$ donc par ce qui précède le lemme II.5, $\sum_{i \in S} s_i \lambda_i^S(0) = l!f(0) = l!d \mod m$. Donc finalement :

$$w^e = x^{4(l!)^2 ed} = x^{4(l!)^2} \mod n$$

d'après la proposition II.6. □

De là, comme $4(l!)^2$ est premier avec e (car on a imposé e premier et plus grand que l), on peut facilement calculer leurs coefficients de Bézout. Notons $4(l!)^2 a + eb = 1$. La signature est ultimement $s = w^a x^b$. En effet, on vérifie que :

$$s^e = (w^e)^a x^{eb} = x^{1-eb} x^{eb} = x \mod n$$

II.3 Étude de la sécurité

Premièrement, expliquons en quoi les protocoles de signature partielles sont bien zero knowledge. Déjà, calculer s_i en tant que logarithme discret en base x' de x_i^2 dans le groupe cyclique $\mathcal{Q}_n$ de cardinal m (on rappelle que x' est presque sûrement un générateur de $\mathcal{Q}_n$) a le même niveau de complexité que chercher à calculer s_i en tant que logarithme discret en base v de v_i dans $\mathcal{Q}_n$. Autrement dit c'est non seulement dur (sécurité en 2^{80} car m est de l'ordre de 1024 bits) mais en plus v_i étant publique, interférer dans le canal afin de connaître x_i n'apporte rien de plus, lorsque l'objectif est de connaître s_i . À noter que c'est cette volonté d'assurer que les calculs se déroulent dans $\mathcal{Q}_n$, là où le problème du logarithme discret est difficile, qui nous à pousser à considérer x_i^2 plutôt que x_i lors de la signature partielle du joueur i . Précisément, imaginons que l'adversaire a réussi à envoyer (x_i, z, c) où $c = H'(v, x', v_i, x_i^2, v^z v_i^{-c}, x'^z x_i^{-2c})$. Les arguments étant dans $\mathcal{Q}_n$, le groupe engendré par v , on note $x' = v^\alpha$, $v_i = v^{s_i}$, $x_i^2 = v^\beta$, $v^z v_i^{-c} = v^\gamma$, $x'^z x_i^{-2c} = v^\delta$. On a donc $v^\gamma = v^{z-s_i c}$ et $v^\delta = v^{\alpha z - \beta c}$ autrement dit $\gamma = z - s_i c \mod m$ et $\delta = \alpha z - \beta c \mod m$ et donc

$$\alpha \gamma - \delta = c(\beta - \alpha s_i) \mod m$$

Deux cas alors : Ou bien $\beta - \alpha s_i$ est inversible modulo m . Mais alors $H'(v, v^\alpha, v^{s_i}, v^\beta, v^\gamma, v^\delta) = \frac{\alpha\gamma - \delta}{\beta - \alpha s_i} \pmod m$ ce qui est quasiment impossible dans le modèle de l'oracle aléatoire (une fonction de hachage prends ses sorties uniformément et indépendamment de ses arguments). Ou bien $\beta - \alpha s_i$ n'est pas inversible modulo m . Or la probabilité que ce soit divisible par un diviseur de m et pas l'autre est négligeable (sinon m serait facilement factorisable, et donc n aussi). On peut donc supposer que $\beta - \alpha s_i = 0 \pmod m$. Mais α étant presque sûrement inversible modulo m , cela signifie que l'adversaire connaît s_i . Conclusion : envoyer un bon (x_i, z, c) implique (presque sûrement) de connaître s_i , ce qui valide la robustesse du protocole.

Ensuite assurons nous que la signature est bien zero knowledge, c'est-à-dire que la connaissance de $z = s_i c + r$ ne donne aucune information. Pour cela on a besoin du lemme suivant :

Lemme II.8. Soit r tiré aléatoirement dans $\{0, \dots, a-1\}$. Alors en moyenne r aura $L(a) - 1$ bits où $L(a)$ est le nombre de bits de a .

Preuve. Soit X la variable aléatoire représentant le nombre de bit de r . On a

$$\mathbb{E}(X) = \sum_{i=1}^{L(a)} i \mathbb{P}(X = i) = \sum_{i=1}^{L(a)} i \frac{2^{i-1}}{2^{L(a)}}$$

Or en dérivant la somme $\sum_{i=1}^{L(a)} x^i$ et évaluant en $x = 2$ on obtient

$$\mathbb{E}(X) = \frac{2^{L(a)+1}(L(a) - 1) + 2}{2^{L(a)+1}} \simeq L(a) - 1$$

□

On remarque que s_i est aléatoire uniformément dans $\{0, \dots, m-1\}$ au vu de la correspondance bijective entre k valeurs de $\{0, \dots, m-1\}$ et le polynôme interpolateur associé, et car les coefficients a_i (et donc le polynôme f) sont aléatoires uniformément. Ainsi par le lemme précédent, s_i a donc en moyenne $L(m) - 1$ bits. Or

$$\frac{n}{4} - m = \frac{(2p' + 1)(2q' + 1)}{4} - p'q' = \frac{p' + q'}{2} + \frac{1}{4}$$

Donc

$$\lfloor \frac{n}{4} \rfloor - m = \frac{p' + q'}{2} \simeq \sqrt{n}$$

Donc finalement s_i a en moyenne $L(n) - 3$ bits. La fonction de hachage H étant supposé parfaitement répartie dans l'ensemble des nombres à moins de L_1 bits, c a environ $L_1 - 1$ bits. Donc $s_i c$ a en moyenne $L(n) + L_1 - 4$ bits. Tandis que r est tiré uniformément dans l'ensemble des nombres à moins de $L(n) + 2L_1$ bits, et a donc en moyenne $L_1 + 3$ bits de plus que le produit $s_i c$. L_1 étant non négligeable (par exemple $L_1 = 128$), sommer le nombre aléatoire r permet de brouiller uniformément $s_i c$ et donc s_i , rendant le tout zero knowledge.

Deuxièmement démontrons que ce protocole de signature à seuil $(k;l)$ avec $t = k - 1$ tricheurs est sécurisée comme souhaité, c'est-à-dire qu'il est robuste et non forgeable. Pour ce faire, on définit la notion de distance statistique :

Définition II.1. La distance statistique entre deux distributions P_1 et P_2 définies sur un univers Ω est la valeur $\Delta(P_1, P_2) = \frac{1}{2} \sum_{x \in \Omega} |P_1(x) - P_2(x)|$

On va alors chercher à simuler l'attaque de l'adversaire. On note $i_1, \dots, i_{k-1}$ les joueurs corrompus et simplement i le joueur participant non corrompu. On a déjà expliqué que les s_{i_j} étaient uniformément aléatoire dans $\{0, \dots, m-1\}$. Cependant, l'adversaire ne connaît pas m . Il va donc simuler cela en tirant aléatoirement les s_{i_j} dans $\{0, \dots, \lfloor \frac{n}{4} \rfloor - 1\}$. C'est satisfaisant car la distance statistique entre les lois uniformes $U_n := U(\{0, \dots, \lfloor \frac{n}{4} \rfloor - 1\})$ et $U_m := U(\{0, \dots, m-1\})$ est négligeable. En effet :

$$\Delta(U_n, U_m) = \frac{1}{2} \left(\sum_{i=0}^{m-1} \left| \frac{1}{m} - \frac{1}{\lfloor \frac{n}{4} \rfloor} \right| + \sum_{i=m}^{\lfloor \frac{n}{4} \rfloor - 1} \left| 0 - \frac{1}{\lfloor \frac{n}{4} \rfloor} \right| \right) = \frac{1}{2} \left(1 - \frac{m}{\lfloor \frac{n}{4} \rfloor} + 1 - \frac{m}{\lfloor \frac{n}{4} \rfloor} \right)$$

Donc finalement, $\Delta(U_n, U_m) = \frac{\frac{p'+q'}{2}}{\lfloor \frac{n}{4} \rfloor} = O\left(\frac{1}{\sqrt{n}}\right)$.

À ce stade l'adversaire a donc choisi les valeurs des s_{i_j} et cherche à connaître s_i . Il considère alors $S = \{0, i_1, \dots, i_{k-1}\}$. En lisant les données qui passent dans le canal lors de la signature, il peut récupérer $x, y \in \mathbb{Z}_n^\times$ tel que $x = y^e$. Il peut, tout comme n'importe qui connaissant les i_j et i pourrait, également calculer les $\lambda_{i_j}^S(i)$ et $\lambda_0^S(i)$. Ainsi il peut de lui même calculer x_i via :

$$x_i = x^{2l!s_i} = x^{2(\lambda_0^S(i)s_0 + \sum_{i_j} \lambda_{i_j}^S(i)s_{i_j})} = y^{2(\lambda_0^S(i) + e \sum_{i_j} \lambda_{i_j}^S(i)s_{i_j})}$$

d'après le lemme II.5 et le théorème de Fermat avec $\lambda(n) = 2m$ la fonction indicatrice de Carmichael de n . Ainsi l'adversaire ne gagne aucune information sur le secret de partage s_i en connaissant x_i puisqu'il aurait pu lui même le simuler. De plus cette formule met en lumière que pour $t = k - 1$ tricheurs, trouver une racine e -ième de x (c'est-à-dire résoudre le problème RSA) équivaut à forger la signature.

III Protocole de chiffrement à seuil El Gamal

III.1 Présentation du protocole

On se place dans $\mathbb{F}_q$ avec q une puissance de nombre premier. Un générateur $g \in \mathbb{F}_q^\times$ est donc d'ordre $q - 1$. Comme pour le protocole RSA on va faire de l'interpolation modulo l'ordre de nos éléments. On a donc envie que $q - 1$ soit premier. En particulier q doit être pair et donc prend nécessairement $q = 2^n$ avec $n \geq 1000$ pour que le problème du logarithme discret soit difficile et tel que $q - 1 = 2^n - 1$ soit (un nombre de Mersenne) premier. Par exemple $n = 1279$ assure ces deux critères. Soit donc un tel n , un tel $q = 2^n$ et $g \in \mathbb{F}_q^\times$ un générateur.

Remarque. À ce stade on voit déjà les deux grosses différences avec le protocole RSA, qui rendront plus simple le protocole EL Gamal. Premièrement $q - 1$ est premier contrairement à l'exposant $\lambda(n) = 2m$ dans le protocole RSA. On a donc ici ni à se soucier d'élever les éléments au carré lors du partage de secret par exemple, ni à se demander si les dénominateurs apparaissant lors de l'interpolation sont inversibles. Deuxièmement $q - 1$ est publique contrairement à m . Ainsi tout le monde, en particulier le coordinateur désigné, est capable d'inverser les dénominateurs apparaissant lors de l'interpolation modulo $q - 1$. La multiplication par $l!$ qui apparaissait pour la signature RSA n'est donc plus nécessaire lors de la génération de secret partiel.

Le protocole à seuil $(k; l)$ se déroule alors comme suit :

Le dealer : Le dealer tire aléatoirement un entier $a_0 = d$ entre 0 et $q - 2$ qui sera sa clé secrète et comme pour le premier protocole, il tire $k - 1$ entiers $a_1, \dots, a_{k-1}$ entre 0 et $q - 2$ qui forment le polynôme

$$f = \sum_{i=0}^{k-1} a_i X^i \in \mathbb{Z}[X]$$

Il calcule alors pour tout i entre 1 et l la valeur $s_i = f(i) \mod q - 1$ qui constitue la clé secrète du joueur i . Finalement, la clé publique est la famille $(\mathbb{F}_q, g, g^d)$.

L'expéditeur : Disons que Bob veut chiffrer un message $M \in \mathbb{F}_q^\times$. Il tire aléatoirement un entier b entre 0 et $q - 2$ qu'il garde secret. Il envoie alors le couple $(x, y) := (g^b, g^{bd} M)$.

Déchiffrement partielle : On note $S = \{x_1, \dots, x_k\}$ les joueurs. On part du principe que $t \leq k - 1$ de ces joueurs sont potentiellement corrompus c'est-à-dire que ces adversaires possèdent leurs t clés secrètes et ont deux objectifs : connaître les autres clés secrètes et être capable de déchiffrer des messages en ne réunissant que les joueurs corrompus. Soit $i \in \{1, \dots, k\}$, pour le joueur x_i , le déchiffrement partielle consiste simplement à calculer $z_i = x^{s_i} \in \mathbb{F}_q^\times$. On remarque que ce protocole, tout comme le précédent, est non interactif.

Mise en commun : Admettons que le coordinateur désigné ait reçu les k partages valides z_i . Il va alors calculer $z = \prod_{i=1}^k z_i^{L_i^S(0)}$ où $L_i^S = \frac{\prod_{u \in S \setminus x_i} X - u}{\prod_{u \in S \setminus x_i} x_i - u} \in \mathbb{Z}_{q-1}[X]$ est le polynôme du lemme II.5.

Il a alors en fait calculer $z = g^{bd}$. En effet :

$$z = \prod_{i=1}^k z_i^{L_i^S(0)} = \prod_{i=1}^k x^{s_i L_i^S(0)} = \prod_{i=1}^k g^{bs_i L_i^S(0)} = g^{b \sum_{i=1}^k s_i L_i^S(0)}$$

Or $\sum_{i=1}^k s_i L_i^S(0) = f(0) = d \pmod{q-1}$ d'après des résultats classiques d'interpolation dans le corps $\mathbb{Z}_{q-1}$ (c.f le paragraphe précédent le lemme II.5 en remplaçant m par $q-1$). Finalement le coordinateur calcule $M = yz^{-1}$ et retrouve le message.

III.2 Etude de la sécurité

La création du polynôme f étant la même que dans le protocole RSA à seuil, la preuve de la bonne uniformité de la répartition des clés secrètes s_i est la même. Ainsi la connaissance de certains s_i (ceux des corrompus par exemple) ne donne aucune information sur les autres s_i . L'adversaire n'a donc pas d'avantage par rapport à une personne tierce ne faisant que de lire les données émises à chaque étape du protocole. Ensuite on remarque facilement que trouver M à partir de $(x, y) = (g^b, g^{bd}M)$ et de g^d revient exactement à retrouver g^{bd} à partir de g^b et de g^d . Autrement dit il s'agit de résoudre le problème Diffie-Hellman (en base g). De même retrouver s_i à partir de $z_i = x^{s_i}$ est le problème du logarithme discret (en base $x = g^b$) ce qui est tout aussi dur car $\text{ord}(g^b) = \text{ord}(g) = q-1$. En conclusion, l'adversaire ne peut ni facilement retrouver les s_i ni déchiffrer le message seul.

IV Implémentation

Premièrement, les deux protocoles ayant de nombreuses similarités, il a été choisi d'utiliser (presque) les mêmes fichiers de fonctions annexes pour les deux programmes. La différence ce fait donc sur les fichiers contenant les différentes fonctions *main* et à la compilation.

Aussi, le contexte de ce projet fait intervenir différentes "personnes" ayant des droits et des rôles différents. Ceci impose certaines contraintes dans l'implémentation. En effet, demander à un utilisateur de signer un message pour lui-même montre un sérieux manque d'intérêt. Aussi, il a été choisi de simuler l'existence des différentes personnes par la création de dossiers. Ainsi, il est plus facile de contrôler les accès à certaines données.

Commençons par rappeler la hiérarchie des différents rôles (des deux protocoles) :

- **Le dealer** : il fournit et a donc accès aux clés de l'ensemble des joueurs. Son rôle se limite cependant à définir le nombre de joueurs nécessaires à la signature/chiffrement et à générer les clés. Il n'intervient plus ensuite.
- **Le joueur** : il a accès à ses clés et peut connaître les clés de vérifications des autres joueurs. En pratique, il se contente de fournir sa signature partielle au coordinateur durant la signature du message.
- **Le coordinateur** : il demande à chaque joueur impliqué sa signature / son déchiffrement partiel(le) ainsi que la preuve d'exactitude pour la signature. Pour la signature, il vérifie que les signatures partielles fournies sont exactes. Dans les deux programmes, il combine les informations reçues pour totalement signer/déchiffrer le message. Pour la signature, il termine en envoyant le message signé au vérifieur.
- **Le vérifieur** : il n'a accès qu'au message accompagné de la signature. Il vérifie et informe si le message est alors bien signé.
- **L'expéditeur** : il envoie son message chiffré et les informations nécessaires au déchiffrement au coordinateur.

Une autre problématique majeure a été l'implémentation de la structure du corps $\mathbb{F}_{2^n}$. Pour ce faire, on choisit un polynôme primitif P sur $\mathbb{F}_2[X]$ de degré n et on travaille dans le quotient $\mathbb{F}_2[X]/(P)$. De plus, on représente nos polynômes $a_iX^i + \dots + a_1X + a_0 \in \mathbb{F}_2[X]$ par un entier de GMP de valeur $a_i2^i + \dots + a_12 + a_0 \in \{0, \dots, 2^{i+1} - 1\}$ et on programme l'ensemble des fonctions nécessaires à la structure selon cette représentation.

Structure du code : on regroupe les fonctions par rôle, ce qui nous donne cinq fichiers : *Dealer_functions.c*, *Coordinator_functions.c*, *Players_functions.c*, *Sender_functions.c* et *Verifier_functions.c*. Auxquels s'ajoutent *General_function.c* qui contient des fonctions n'étant pas nécessairement associées à un rôle particulier et *RSA_Threshold.c/El_Gamal_Threshold.c* qui contiennent toute les fonctions principales mettant en marche leur programme respectif, ainsi que la fonction *main* associée. Plus précisément, la fonction principale contenue à la fois dans *RSA_Threshold.c* et *El_Gamal_Threshold.c* est la fonction *ask_action* qui permet à l'utilisateur

de choisir l'action à effectuer parmi :

Signature RSA	Chiffrement El Gamal
• Générer de nouveaux joueurs/clés • Signer un message • Vérifier une signature • Supprimer les fichiers des joueurs • Quitter le programme	• Générer de nouveaux joueurs/clés • Envoyer un message chiffré • Déchiffrer un message • Supprimer les fichiers des joueurs • Quitter le programme

L'option "Supprimer les fichiers des joueurs" ayant pour objectif de supprimer les fichiers et dossiers des joueurs résiduels au cas où l'on diminuerait le nombre de joueurs total. Enfin, on accompagne ces fichiers de *Keccak.c* fourni dans l'une des séances de cours de programmation en C. Ce fichier contient tout le nécessaire pour construire des fonctions de hachage. Cependant, il est bon de remarquer que la fonction *Keccak* utilisée dans notre programme est optimale lorsque la *capacité* correspond à deux fois le nombre de bits de sortie. Or, ceci correspondrait à une *capacité* de 2048 puisque notre sortie est de 1024 bits, ce qui est impossible puisque la somme de la *capacité* et du *taux* doit être de 1600. Des concessions ont donc été faites concernant nos fonctions de hachage, cet aspect ne faisant pas partie du cœur du projet. Enfin, les différents programmes sont fournis avec les clés du dealer. Elles regroupent toutes les propriétés nécessaires au bon fonctionnement des programmes. La génération de ces clés a été faite de manière annexe et n'est pas comprise dans ce programme. Ainsi, il est conseillé d'utiliser les clés par défaut ou d'avoir sous la main d'autres clés adéquates.

Compilation : on utilise un Makefile qui permet de compiler les deux programmes de ce projet. Les commandes "*make Signature_RSA*" et "*make Chiffrement_El_Gamal*" compile chaque programme respectif. On peut aussi simplement utiliser "*make*" qui compile les deux programmes à la fois. À ces commandes s'ajoute "*make clean*" afin de supprimer les fichiers objets et les exécutable. Les options de compilation utilisées sont *-Wall* pour avoir l'ensemble des warnings de compilation, *-g* pour l'utilisation de gdb à des fins de débogage, *-fsanitize=address* pour déterminer et éliminer les fuites mémoire et enfin *-lgmp* pour l'utilisation de la bibliothèque GMP.

Critique sur l'implémentation : le programme n'étant pas très coûteux en ressources et ayant pour vocation d'être utilisé avec de petites valeurs (par exemple $l = 5$ et $k = 3$), le principal objectif dans la programmation a été la simulation du transfert d'informations entre les joueurs. Aussi, même si des efforts ont été fait pour optimiser certains calculs, il est certain que la complexité de nombreuses fonctions puisse être réduite.

Enfin, il est d'usage en programmation de faire de la *gestion d'erreurs*. Par exemple lors d'allocation mémoire, rien n'a été prévu s'il l'espace mémoire nécessaire au bon fonctionnement du programme est insuffisant. Ce genre de considérations ne sont pas primordiales dans le contexte de ce projet, mais constituent des pistes d'améliorations principales.

V Performances de l'implémentation

On veut évaluer la performance de notre implémentation pour $1 \leq k \leq 50$ avec un $l = 100$ fixé. Ainsi, pour chaque k , on effectue 1000 signatures et 100 déchiffrements afin d'estimer la durée d'une signature/chiffrement en millisecondes. On obtient les résultats suivants :

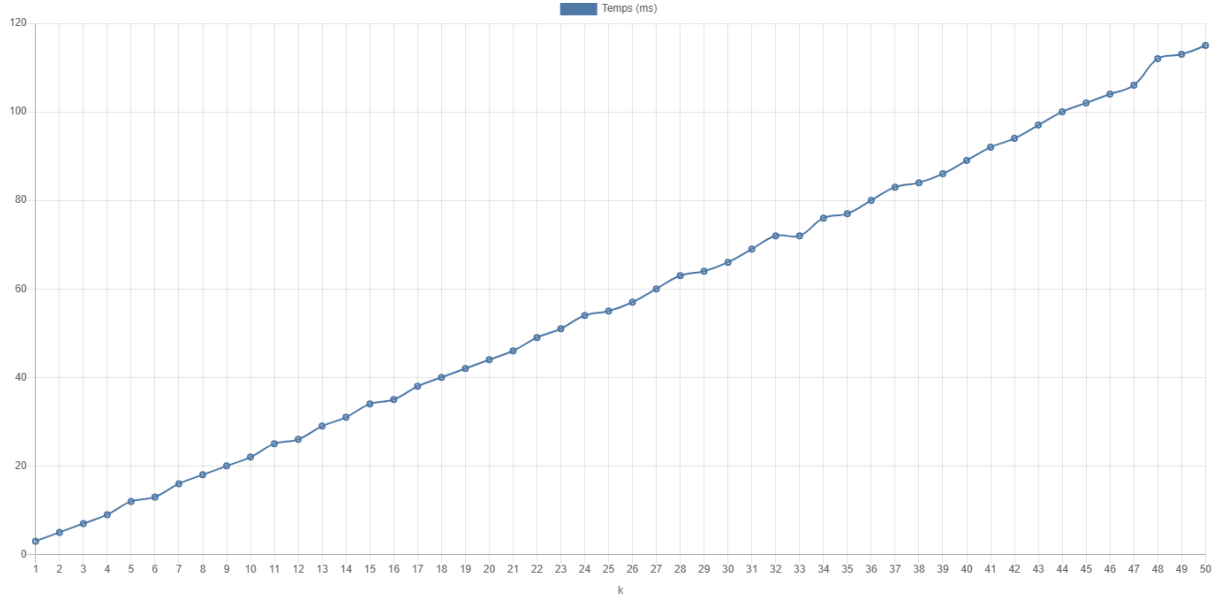


FIGURE 1 – Temps d'une signature RSA à seuil en fonction de k pour $l = 100$

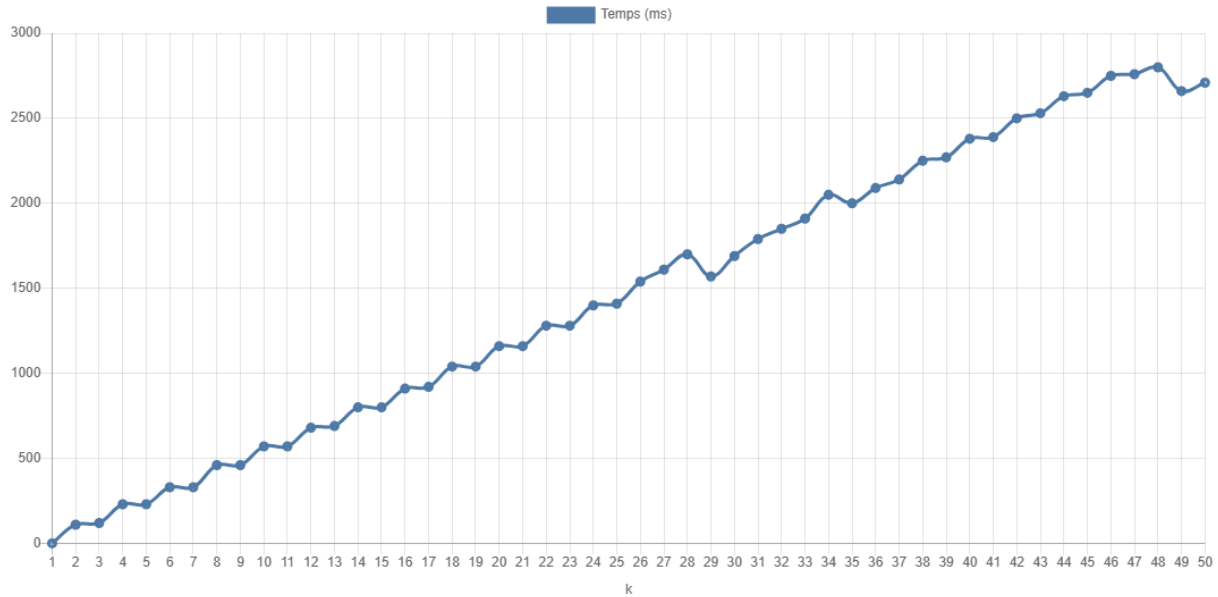


FIGURE 2 – Temps d'un déchiffrement El Gamal à seuil en fonction de k pour $l = 100$

Pour chacune de ces simulations, on peut effectuer une régression linéaire afin d'estimer l'évolution globale du temps de calcul.

Signature : le temps t semble suivre une évolution d'équation $t(k) = 2,26k - 0.60$ avec un $R^2 = 0.999$, ce qui correspond donc à un $\mathcal{O}(k)$.

Déchiffrement : le temps t semble suivre une évolution d'équation $t(k) = 59,0k - 41,6$ avec un $R^2 = 0.996$, ce qui correspond donc à un $\mathcal{O}(k)$.

On constate premièrement un facteur supérieur à 10 entre les deux protocoles pour le temps de calcul. Ceci est certainement causé par les calculs faits dans le corps $\mathbb{F}_{2^n}$ et l'implémentation des fonctions associées qui pourrait être optimisées. En effet, l'ensemble des calculs faits pour la signature utilise des fonctions de la bibliothèque GMP qui sont déjà très optimisées.

Ensuite, on peut remarquer dans les résultats obtenus pour le déchiffrement que certaines valeurs vont par paires. En effet, les temps de calculs sont identiques pour certains couples de valeurs de $k : (2, 3), (4, 5), (6, 7), \dots$. Les raisons d'un tel phénomène n'ont pas été identifiées et mériteraient davantage de tests.

Enfin, il est bon de noter que ces simulations n'ont pas été faites dans des situations optimales puisque l'ordinateur faisant les calculs était utilisé pendant les simulations.

Bibliographie

- [1] Victor Shoup. Practical threshold signatures. In *Proceedings of the 19th International Conference on Theory and Application of Cryptographic Techniques*, EUROCRYPT'00, page 207–220, Berlin, Heidelberg, 2000. Springer-Verlag.
- [2] Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In Gilles Brassard, editor, *Advances in Cryptology — CRYPTO' 89 Proceedings*, pages 307–315, New York, NY, 1990. Springer New York.